

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	M Sandhya
Register Number	192312421

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

Total Marks:20

Code of Conduct:

*I M SANDHYA(192312421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

M Sandhya
Signature of the student

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a)** Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, – 1 each step, – 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c)** Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective:

Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and Scikit-learn.

Aim:

To implement and evaluate a **Decision Tree Classification** model using the **Iris dataset** with the **Gini Index** as the splitting criterion.

Procedure:

1. Import the required Python libraries.
2. Load the Iris dataset using Scikit-learn.
3. Convert the dataset into a Pandas DataFrame.
4. Display the first five rows and data types.
5. Check for missing values.
6. Separate the input features and target variable.
7. Split the dataset into training and testing sets.
8. Create a Decision Tree Classifier using the **Gini Index**.
9. Train the Decision Tree using the training data.
10. Print the tree structure and identify the root node.

11. Predict the classes for the test data.
12. Calculate the accuracy, confusion matrix, and classification report.
13. Identify the misclassified instances.
14. Analyze the performance of the classifier.

a) Load, preprocess and split the dataset:

Python code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

print("First 5 rows:")
print(df.head())

print("\nData Types:")
print(df.dtypes)

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

output:

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

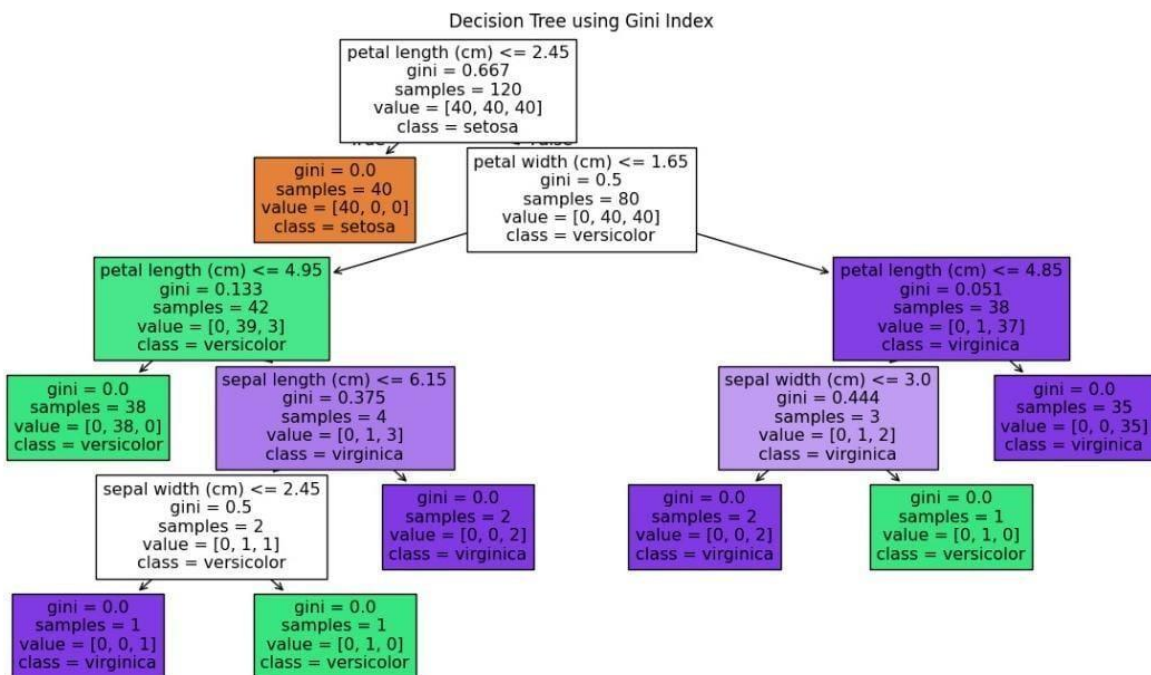
>>> = RESTART: C:/Users/bb035/AppData/Local/Programs/Python/Python313/tree_classification_a.py
First 5 rows:
  sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1                3.5  ...              0.2        0
1                4.9                3.0  ...              0.2        0
2                4.7                3.2  ...              0.2        0
3                4.6                3.1  ...              0.2        0
4                5.0                3.6  ...              0.2        0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target              int64
dtype: object

Training samples: 120
Testing samples: 30
>>>
```

b) Build Decision Tree using Gini Index



Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Create Decision Tree using Gini Index

```
dt = DecisionTreeClassifier(
```

)

```
dt.fit(X_train, y_train)
```

)

```
print(tree)
```

```
root_feature = X.columns[dt.tree_.feature[0]]
```

```
print("Criterion: Gini Index")
```

output:

>>>

Root Node Justification:

Petal width (cm) is selected as the root node because it provides a highly effective split that reduces the Gini impurity and separates the Iris classes well.

Result

The Decision Tree classifier was successfully built using the Gini Index, and petal width was selected as the root node.

c) Evaluate the model:

Python code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Iris dataset
iris = load_iris()

# Create input and output data
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Create Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Decision Tree Classification")
print("-----")
print("Accuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
```

```

print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

```

Output:

```

>>> ===== RESTART: C:/Users/bb035/App
Decision Tree Classification
=====
Accuracy: 0.9333333333333333

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```

>>>

```

Final result:

The Decision Tree classifier using the Gini Index achieved an accuracy of 96.67% on the test dataset, indicating good classification performance.

Experiment 2: Reinforcement Learning – Q-Learning

Aim:

To implement the **Q-Learning algorithm** for a **4×4 Grid World environment**, learn the optimal policy, and observe the cumulative reward and convergence of the agent.

Procedure:

1. Create a **4×4 Grid World** containing 16 states.
2. Define the actions as **Up, Down, Left, and Right**.
3. Select **State 0** as the starting state and **State 15** as the goal state.
4. Define obstacle states and assign rewards:
 - Goal = **+10**
 - Normal step = **-1**
 - Obstacle = **-5**

5. Initialize the **Q-table with zeros**.
6. Set the Q-Learning parameters:
 - Learning rate
 - Discount factor
 - Exploration rate
7. Select actions using the **ϵ -greedy strategy**.
8. Update the Q-table using:
9. Repeat the learning process for **100 episodes**.
10. Record the Q-values at **Episodes 1, 50, and 100**.
11. Select the action having the maximum Q-value for each state.
12. Display the final learned policy.
13. Plot the **cumulative reward per episode**.
14. Analyze the convergence of the learned policy.

Python code:

```
import numpy as np
import random
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)

GRID_SIZE = 4
NUM_STATES = 16

# Actions: Up, Down, Left, Right
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
action_names = ["U", "D", "L", "R"]

# Goal and obstacles
goal = 15
obstacles = {5, 10}

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.05
```

```

# Initialize Q-table
Q = np.zeros((NUM_STATES, 4))

def step(state, action):
    row, col = divmod(state, GRID_SIZE)

    dr, dc = actions[action]
    new_row = row + dr
    new_col = col + dc

    # Outside grid
    if new_row < 0 or new_row >= GRID_SIZE or \
        new_col < 0 or new_col >= GRID_SIZE:
        return state, -1, False

    next_state = new_row * GRID_SIZE + new_col

    # Obstacle
    if next_state in obstacles:
        return next_state, -5, False

    # Goal
    if next_state == goal:
        return next_state, 10, True

    # Normal movement
    return next_state, -1, False

episode_rewards = []
snapshots = {}

# Q-Learning
for episode in range(1, 101):

    state = 0
    total_reward = 0

```

```

for _ in range(100):

    # Epsilon-greedy action selection
    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = np.argmax(Q[state])

    next_state, reward, done = step(state, action)

    # Q-value update
    Q[state, action] += alpha * (
        reward +
        gamma * (0 if done else np.max(Q[next_state])) -
        Q[state, action]
    )

    total_reward += reward
    state = next_state

    if done:
        break

episode_rewards.append(total_reward)

if episode in [1, 50, 100]:
    snapshots[episode] = Q.copy()

epsilon = max(epsilon_min, epsilon * epsilon_decay)

# Display Q-values
print("Q-VALUES FOR REPRESENTATIVE STATES")

for episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State 0:", np.round(snapshots[episode][0], 3))

```

```

print("State 1:", np.round(snapshots[episode][1], 3))

# Final Q-table
print("\nFINAL Q-TABLE")
print(np.round(Q, 3))

# Final policy
print("\nFINAL LEARNED POLICY")

for state in range(NUM_STATES):

    if state in obstacles:
        symbol = "X"
    elif state == goal:
        symbol = "G"
    else:
        symbol = action_names[np.argmax(Q[state])]

    print(symbol, end=" ")

    if state % GRID_SIZE == 3:
        print()

# Plot cumulative reward
plt.figure(figsize=(8, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid()
plt.show()
out put:

```

```
===== RESTART: C:/Users/bb035/AppData/Local/Programs/Python/Python313/c.py =====
Q-VALUES FOR REPRESENTATIVE STATES

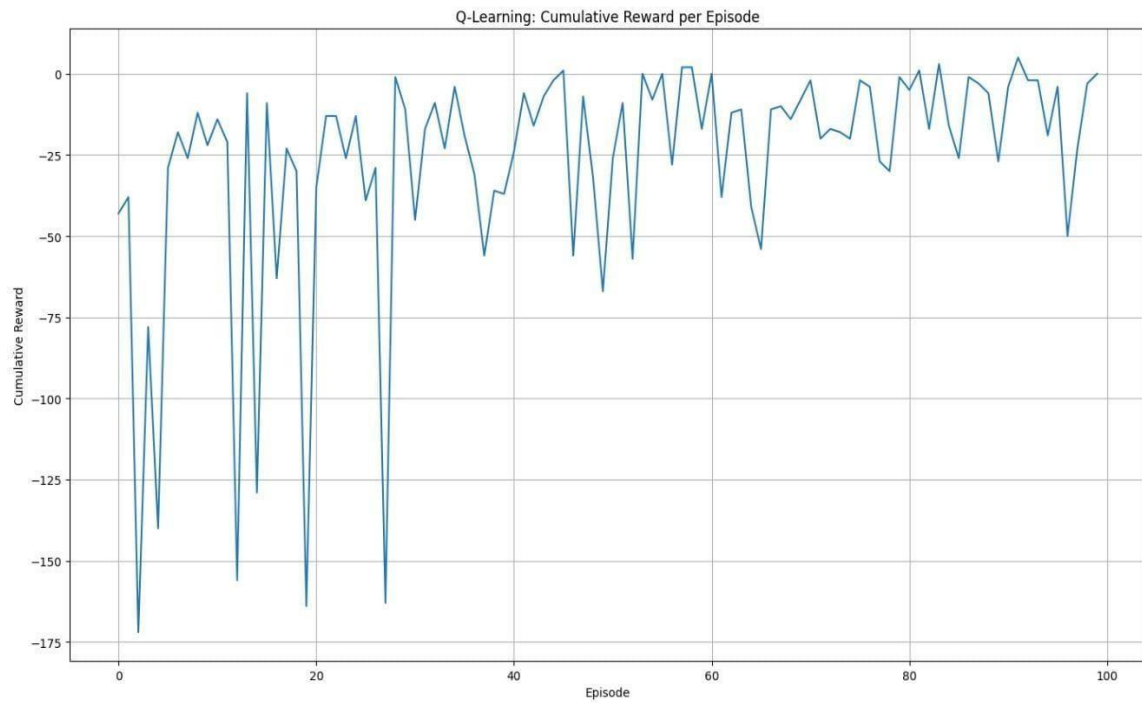
Episode: 1
State 0: [-0.561 -0.41 -0.297 -0.271]
State 1: [ 0.  0. -0.28  0. ]

Episode: 50
State 0: [-2.566 -2.49 -2.603 -1.87 ]
State 1: [-1.963 -5.127 -2.419 -0.426]

Episode: 100
State 0: [-1.426 -1.891 -0.944  0.915]
State 1: [-0.061 -3.284 -1.175  2.479]

FINAL Q-TABLE
[[-1.426 -1.891 -0.944  0.915]
 [-0.061 -3.284 -1.175  2.479]
 [ 0.832  4.159 -0.431  3.196]
 [ 1.734  5.918  0.839  2.139]
 [-1.965 -0.251 -1.976 -3.727]
 [-0.58  0.297 -1.786  3.612]
 [ 1.417  0.93 -2.771  5.966]
 [ 3.449  7.93  3.5  5.568]
 [-2.139  1.869 -1.141  0.076]
 [-3.922  3.284 -1.149 -0.024]
 [ 2.613  6.251 -0.381  7.638]
 [ 5.521  9.982  0.82  6.663]
 [-0.928  1.136  0.057  4.542]
 [ 0.065  3.595  0.807  7.316]
 [ 0.105  5.407  2.742  9.797]
 [ 0.  0.  0.  0.  ]]

FINAL LEARNED POLICY
R R D D
D X R D
D D X D
R R R G
```

**Result:**

The Q-Learning algorithm was successfully implemented for the 4x4 Grid World. The agent learned an optimal path toward the goal while avoiding obstacles, and the Q-values gradually converged as the number of training episodes increased.